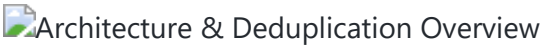
WBPSC AI Mock Tutor — Complete Technical Architecture & Novice-Friendly Engineering Guide


Architecture & Deduplication Overview

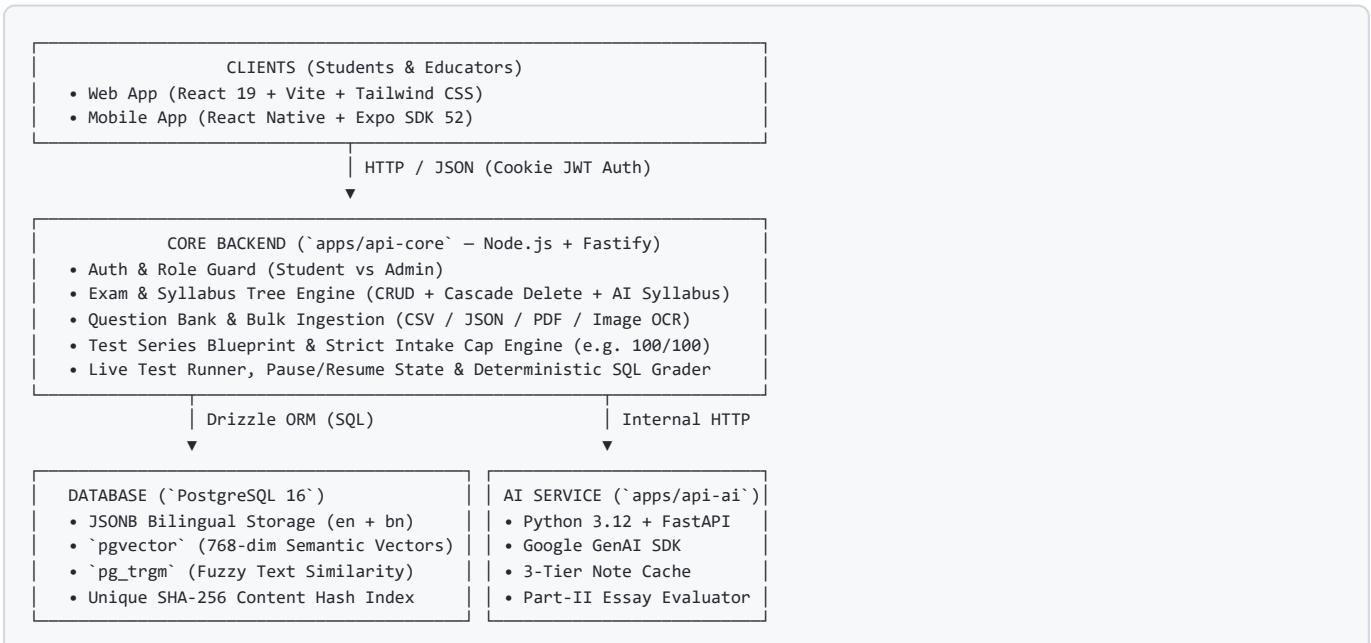
Welcome to the technical documentation for **WBPSC AI Mock Tutor**. This guide is written in **clear, plain English** so that even a beginner developer, student, or non-specialist can understand **what every service does, why each technology was chosen, and how all the moving parts work together behind the scenes**.

---

1. High-Level System Overview (How the App is Structured)

Think of the application as a **modern restaurant**:

1. **The Dining Room (`src/` Web App & `apps/mobile` Android App):** Where students take tests and admins manage exams.
2. **The Head Chef & Cashier (`apps/api-core` — Node.js + Fastify + TypeScript):** Handles login, serves mock tests, grades exams using exact math, manages the syllabus/question bank, and enforces rules (like the 100-question limit).
3. **The Specialist AI Scholar (`apps/api-ai` — Python + FastAPI + Google Gemini):** Writes bilingual study notes, evaluates handwritten/typed descriptive essays, and synthesizes daily current affairs.
4. **The Vault & Memory (`PostgreSQL + pgvector + pg_trgm` & `Redis`):** Stores all users, exams, bilingual questions, paused test progress, and cached AI study notes.
5. **The Shared Rulebook (`packages/shared-types`):** A shared folder of TypeScript definitions and `Zod` validation rules used by both the frontend and backend so they always agree on data formats.



---

2. Technology Stack Explained Simply

Layer	Technology Used	Why We Use It (Plain-English Explanation)
Web Frontend	**React 19, TypeScript, Tailwind CSS, Lucide Icons**	Builds a fast, interactive single-page web interface with clean Bengali (`Noto Sans Bengali`) and English typography.
Mobile App	**React Native (Expo SDK 52, NativeWind v4)**	Allows the same TypeScript logic to run natively on Android phones (`Vivo X200 FE` via Expo Go or `.apk`).
Core API Service	**Node.js 22, Fastify, Drizzle ORM**	`Fastify` is an ultra-fast web server; `Drizzle ORM` lets us write type-safe SQL queries to PostgreSQL without bugs.
AI Microservice	**Python 3.12, FastAPI, `google-genai` SDK**	Connects to Google Gemini models (`gemini-2.5-flash` / `gemini-3.1-flash-lite`) for AI question generation, OCR, translation, and tutoring.
Primary Database	**PostgreSQL (`pgvector` + `pg_trgm`)**	Stores relational tables, bilingual `JSONB` text (`{"en": "...", "bn": "..."}`), fuzzy text trigrams, and 768-dimension vector embeddings.
Speed Cache	**Redis (Upstash / Local Redis)**	Stores frequently requested AI study notes in RAM so they load in `< 5 milliseconds` without calling the AI again.
Validation	**Zod (`packages/shared-types`)**	Checks every incoming request (every form, CSV row, or AI response) before it touches the database.

---

3. How Bilingual (`English` + `বাংলা`) Storage Works

Every exam title, subject, chapter, subtopic, question stem, option (`A`, `B`, `C`, `D`), and explanation is stored in PostgreSQL using a **`JSONB` column** with two keys:

```
{
  "en": "Who was the first Governor-General of Bengal?",
  "bn": "বাংলার প্রথম গভর্নর-জেনারেল কে ছিলেন?"
}
```

Why is this great?

1. **Instant Language Switching:** Because both languages travel together in the same row, when a student clicks **বাংলা (BN)** or **English (EN)** at the top of the screen, the UI switches languages **instantly** without reloading the page or losing their place in the exam.
2. **Automatic English → বাংলা Translation:** When an educator adds a single question or imports a CSV/PDF that only has English text, `apps/api-core/src/lib/aiEnrichment.ts` (`translateAndEnrichQuestionDraft`) automatically calls Gemini to translate the English stem, options, and explanation into natural Bengali before saving to the database.

---

4. How Vector Embeddings (`pgvector`) & Trigram Similarity (`pg_trgm`) Work — And When Each Is Used

Beginners often wonder: *"What is a vector embedding, and when do we use it versus normal SQL?"*

A. What is a Vector Embedding (`pgvector`)?

- A **vector embedding** converts the *meaning* of a paragraph or question into a list of **768 numbers** (for example: `[0.021, -0.044, 0.118, ...]`) stored in the `questions.embedding` column (`vector(768)` in `apps/api-core/src/db/schema.ts`).
- Two texts that mean the same thing—even if they use different words or different languages (English vs. Bengali)—end up with very similar lists of numbers (measured using **Cosine Distance**).

B. When is `pgvector` used?

1. **RAG (Retrieval-Augmented Generation) for AI Study Notes (`apps/api-ai`):** When a student struggles with a subtopic (say, *Fundamental Rights*), the system can retrieve the most semantically relevant PYQs and reference facts from the database using vector similarity so the AI Tutor writes highly accurate, WBPSC-specific notes.
2. **Semantic Search Across the Question Bank:** Finding conceptually related questions across different exams (e.g., finding WBCS Prelims questions that test the same concept as a Clerkship question).

C. What is `pg_trgm` (PostgreSQL Trigram Similarity) and when is it used?

- `pg_trgm` breaks text into 3-letter chunks ("trigrams") and computes a similarity score between `0.0` (completely different) and `1.0` (identical) directly inside PostgreSQL using `similarity(a, b)` .
- **When it is used:** Every time a new question is uploaded or generated by AI, `apps/api-core/src/lib/dedupe.ts` uses `pg_trgm` + option-overlap scoring to catch **reworded duplicate questions** immediately in SQL without paying for an AI embedding call!

---

5. How Repetitive & Duplicate Question Generation Is Prevented (The 3-Layer Shield)

One of the hardest problems with AI question generators is that AI models love to repeat famous textbook questions (like asking *"Who founded the Asiatic Society?"* ten times).

We solve this using a **3-Layer Anti-Duplication Pipeline** (`apps/api-core/src/lib/dedupe.ts` & `contentHash.ts`):


Bilingual Exam & Deduplication Flow

Layer 1: Prompt-Time Stem Avoidance (Before AI Generates Anything)

- Before `api-core` asks Gemini to generate questions for a subtopic or subject, it queries PostgreSQL for up to **80 existing question stems** already in that subtopic/subject (`loadGenerationContext` in `aiGeneration.ts` and `avoidStems` in `ingestion.ts`).
- Those existing stems are injected into the Gemini prompt under `"DO NOT repeat or rephrase any of these existing questions"` .
- In multi-round generation, any question generated in Round 1 is immediately added to the `avoidStems` list for Round 2.

Layer 2: Order-Independent SHA-256 Content Hash (Exact & Shuffled-Option Match)

- Located in `apps/api-core/src/lib/contentHash.ts` (`computeContentHash`).
- What if someone uploads the exact same question, or shuffles Options A, B, C, and D?
- We normalize the English stem (lowercase, strip punctuation, collapse spaces) and **alphabetically sort** the normalized text of options A, B, C, and D:

```
stemNorm || sorted(optA, optB, optC, optD)
```

- We compute a **64-character SHA-256 hash** of this string.
- The `questions` table in PostgreSQL has a **`UNIQUE` index on `content_hash`** (`questions_content_hash_idx`). It is physically impossible for PostgreSQL to insert two identical questions!

Layer 3: Fuzzy Stem + Option-Set Overlap Deduplication (`pg_trgm` + Jaccard Similarity)

- Located in `apps/api-core/src/lib/dedupe.ts` (`nearDuplicateScore` & `findNearDuplicates`).
- What if the AI changes the wording slightly? For example:
  - *Question 1:* `"In which year was the Battle of Plassey fought?"` (Options: `1757, 1764, 1761, 1748`)
  - *Question 2:* `"The historic Battle of Plassey took place in which year?"` (Options: `1757, 1764, 1761, 1750`)
- Our `nearDuplicateScore(a, b)` function checks **three signals**:
  1. **Trigram Stem Similarity (`stemSim`):** Uses PostgreSQL `similarity()` (or in-memory trigram Dice similarity). If `stemSim >= 0.75` , it is rejected as a near-duplicate.
  2. **Combined Stem + Option Overlap:** If at least **2 out of 4 options are identical** (`optionOverlap >= 0.50`) AND the stem is at least `55%` similar (`stemSim >= 0.55`), it is rejected!
  3. **Same Answer-Set Rewording:** If **3 out of 4 options are identical** (`optionOverlap >= 0.75`) AND the stem shares at least `35%` similarity, it is also caught and discarded!

---

6. How Exam Evaluation Works (Zero-AI Deterministic SQL Grading)

 Live Exam Runner

Golden Rule: We **never** use an AI model to grade multiple-choice questions. AI models are slow and can make arithmetic mistakes. Instead, `POST /exams/submit` (`apps/api-core/src/routes/exam.ts`) grades an entire 100-question or 200-question exam in **under 25 milliseconds** inside a **single atomic PostgreSQL transaction (`db.transaction`)**:

Step-by-Step Grading Pipeline:

1. **Load Marking Rules:** Reads `marksPerCorrect` (e.g., `+1.00`), `negativeMarksPerWrong` (e.g., `-0.25` or `-0.33`), and `estimatedCutoffMarks` from the Test Series (or Exam).
2. **Fetch Official Answer Keys in One Query:** Joins `questions` , `subjects` , `chapters` , and `subtopics` in a single SQL query to get `correctOption` (`A` / `B` / `C` / `D`) and `expectedTimeSeconds` for every question in the submission.
3. **Deterministic Score Calculation:**
   - **Correct Answer:** Adds `+marksPerCorrect` (e.g. `+1.00`).
   - **Wrong Answer:** Deducts `-negativeMarksPerWrong` (e.g. `-0.25`).
   - **Unanswered / Skipped:** Awards `0.00` marks (no negative penalty).
   - **Final Net Score:** `rawPositiveScore - negativePenalty` .
4. **Clean Up Any Paused State & Save Attempt:** If the test was resumed from a paused/postponed state (`status = 'in_progress'`), the temporary paused row is replaced with the final `status = 'submitted'` record in `test_attempts` and all individual responses are bulk-inserted into `attempt_answers` .
5. **Atomic Subtopic Mastery Upsert (`user_subtopic_mastery`):**
   - For every subtopic tested in the paper, PostgreSQL executes an `INSERT ... ON CONFLICT (user_id, subtopic_id) DO UPDATE` query.
   - It cumulatively updates `total_encountered` , `total_correct` , `total_incorrect` , `accuracy_percentage` , and `avg_time_per_question_seconds` .
   - It computes a `weakness_priority_score` :

     ```
     Priority Score = (100 - Accuracy%) + TimeOverrunPenalty (0 to 50)
     ```

   - It assigns the student's **Mastery Band**:
     - `< 50%` → `critical_weakness`
     - `50% - 69%` → `needs_revision`
     - `70% - 84%` → `proficient`
     - `>= 85%` → `exam_ready`

---

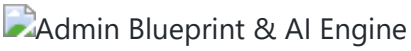
7. How Pause / Postpone & Resume Ongoing Exam Works

When a student is taking a test and clicks "**Pause / Postpone Test**":

1. **Frontend (`src/App.tsx` → `handlePauseMockTest`):** Collects the student's current `elapsedTotalSec`, every question's `selectedOption` (`A` / `B` / `C` / `D` or `null`), whether the question was `marked_for_review`, and the per-question `timeSpentSeconds`.
2. **Backend (`POST /exams/pause` in `apps/api-core/src/routes/exam.ts`):**
   - Saves a row in `test_attempts` with `status = 'in_progress'` and `totalTimeSpentSeconds`.
   - Saves all ordered questions and the student's current answers/bookmarks in `attempt_answers` with `position = 0..N-1`.
3. **Resuming Later (`GET /me/paused-attempts` in `apps/api-core/src/routes/me.ts`):**
   - When the student logs in later, the app fetches their `in_progress` attempts.
   - Clicking "**Resume / Continue Exam**" restores the exact question list, puts their selected options and review bookmarks back on the screen, jumps to the first unanswered question, and resumes the countdown timer from `durationMinutes * 60 - totalTimeSpentSeconds`!
4. **Attempt Count Tracking (`GET /exams` & `GET /exams/:examId/series`):**
   - Counts all `submitted` attempts by that user for each exam and each test series (`myAttemptCount`), calculates `myBestScore` and `myLastScore`, and checks for any `in_progress` paused attempt (`myPausedAttemptId`).

---

8. How Setting Up Exams, Syllabus, Questions & Test Series Works


Admin Blueprint & AI Engine

Step 1: Setting Up an Exam & Its Syllabus (`Admin Console → 2. Exams & Syllabus`)

- **Exam Creation:** Admin picks an official WBPSC preset (e.g., *WBPSC Clerkship*, *WBCS Prelims*, *Food SI*) or enters custom settings (`totalQuestions`, `durationMinutes`, `marksPerCorrect`, `negativeMarksPerWrong`, `estimatedCutoffMarks`).
- **Syllabus Hierarchy (`Subjects → Chapters → Subtopics`):**
  - Stored in three linked PostgreSQL tables: `subjects` → `chapters` → `subtopics`.
  - Admins can build this tree in **3 ways**:
    1. ✨ **Generate Syllabus with AI (`POST /admin/syllabus/:examSlug/generate-ai`):** Gemini generates a complete bilingual syllabus tree for the chosen subjects and skips any duplicate headings.
    2. **Manual CRUD (`+ Add Subject / Topic`, Edit, Delete):** Non-technical forms with pre-populated WBPSC subject lists. Includes **Cascade Delete** (`?cascade=true`) that safely removes child chapters, subtopics, and questions inside a transaction if an admin deletes an old subject.
    3. **Excel / CSV Upload:** Bulk import a syllabus spreadsheet.

Step 2: Building the Question Bank (`Admin Console → 4. Question Bank`)

Admins have **4 pipelines** to add questions into the `questions` table:

1. **Single Question Entry (`+ Add One Question`):** Admin types the question in English. For Bengali, they either click "✨ **Auto-Translate to বাংলা**" or leave Bengali blank so the backend translates it automatically on save.
2. **Upload PDF or Scanned Image of a Question Paper (`POST /admin/ingestion/extract-media`):** Uses Gemini Multimodal Vision to read a PDF or photo of a printed question paper, extract every MCQ, translate missing Bengali text, auto-create/match the Subject → Chapter → Subtopic in the database, and insert the questions.
3. **Upload Excel / CSV / JSON (`POST /admin/ingestion/upload`):** Bulk imports hundreds of questions with automatic deduplication and AI enrichment for any blank Bengali or explanation fields.
4. **Generate with AI (`POST /admin/ingestion/generate`):** Admin selects an Exam, Subject, Chapter, Subtopic, and picks **10, 15, 20, 25, 30, 40, or 50 questions** from the dropdown.

Step 3: Creating a Test Series & Strict Intake Cap Enforcement (`Admin Console → 5. Test Series`)

When creating a Test Series (e.g., *Clerkship Mock Test 1 - 2026* where the exam's total intake is **100 questions**):

- **Two Clean, Isolated Creation Modes:**
  - **Option 1 (Subject-Wise Blueprint):** Admin ticks subjects and sets how many questions come from each subject (e.g., `GS: 40, Arithmetic: 30, English: 30 = 100`). They then choose either **1-Click AI Full Set Generation** or **Select from Question Bank**.
  - **Option 2 (Full Question Paper Upload):** Admin uploads a complete 100-question or 200-question paper (CSV/JSON/PDF/Image) right inside the Create Series form; the backend imports the questions and builds the series automatically.
- **How the Strict `100/100` Cap Works (Preventing `135/100` Overflow):**
  1. **Blueprint Creation & Update (`POST /admin/series/exams/:id` & `PUT /admin/series/:id/blueprint`):** Validates that the sum of subject question counts never exceeds `exam.totalQuestions` (e.g., `100`).
  2. **Question Assignment (`assignQuestions` in `apps/api-core/src/lib/series.ts`):** Checks both the individual subject's vacancy (`slot.targetCount`) AND the overall series cap (`paths.length >= state.rosterTotal`). Once a 100-question series has 100 questions, any 101st question is rejected with `"Test series has reached its maximum question limit (100/100)"` .
  3. **1-Click AI Set & Import Set (`POST /admin/ingestion/generate-full-set` & `POST /admin/series/:id/import-set`):** If a series already has 35 questions assigned out of 100, running 1-Click AI Set or Import Set only fills the **remaining 65 vacancies** (`100 - 35 = 65`) and never inflates the series blueprint to `135/100` . An automatic `enforceSeriesQuestionCap()` guard also trims any legacy overflow rows so `selectedCount` can never exceed `totalQuestions` .

---

9. How the 3-Tier AI Cache Saves Money & Runs Fast

Calling an AI model for the exact same study note 1,000 times for 1,000 students would be slow and expensive. Instead, `apps/api-ai` uses a **Deterministic Cache Key**:

```
Cache Key = "{exam_slug}:{subtopic_slug}:{language}"
Example   = "wbcs-prelims:fundamental-rights:bn"
```

1. **Tier 1 — Redis Memory Cache (`L1`):** Checked first (`< 2 ms`). If found, returned immediately.
2. **Tier 2 — PostgreSQL `ai_study_notes_cache` Table (`L2`):** Checked if Redis misses (`< 10 ms`). If found, it warms Redis and returns the note.
3. **Tier 3 — Google Gemini API (`L3`):** Called **only once** on a complete cache miss. The generated bilingual note is then saved permanently in PostgreSQL and Redis so every future student gets an instant cache hit!

---

10. Quick Reference: Where Everything Lives in the Codebase

Folder / File Path	What It Does
`/src/App.tsx`	Main student web application (Login, Mock Test Center, Live Test Runner with Pause/Resume, Weakness Radar, Part-II Evaluator, Daily CA).
`/src/components/ExamLibrary.tsx`	Exam Category cards & Test Series cards showing Attempt Counts (`Attempted X times`), Best/Last Scores, and `Resume Exam` buttons.
`/src/admin/ExamSeriesManager.tsx`	Admin Console Tabs `2. Exams & Syllabus` and `5. Test Series` (Statistics Banner, 2 Isolated Creation Modes, Cascade Delete).
`/src/admin/SeriesToolsModals.tsx`	Admin modals for **A4 Print PDF & Excel Export**, **Whole Question Set Import**, and **1-Click AI Full Set Generator**.
`/src/admin/QuestionBankAdmin.tsx`	Admin Console Tab `4. Question Bank` (Single Question with Auto-Translate, CSV/JSON Upload, PDF/Image OCR Upload, AI Topic Generator).
`/apps/api-core/src/db/schema.ts`	Complete PostgreSQL database schema (`exams` , `subjects` , `chapters` , `subtopics` , `questions` , `exam_series` , `series_roster_slots` , `series_questions` , `test_attempts` , `attempt_answers` , `user_subtopic_mastery`).
`/apps/api-core/src/routes/exam.ts`	Student exam endpoints (`GET /exams` , `GET /exams/:id/series` , `POST /exams/pause` , `POST /exams/submit` , `POST /exams/remedial-mock`).
`/apps/api-core/src/routes/me.ts`	Student history, detailed answer review, subtopic mastery radar, and paused attempts (`GET /me/paused-attempts`).
`/apps/api-core/src/lib/series.ts`	Test Series blueprint validation, strict question cap enforcement (`enforceSeriesQuestionCap`), and question assignment logic.
`/apps/api-core/src/lib/dedupe.ts`	Fuzzy trigram (`pg_trgm`) and option-overlap near-duplicate question detector.
`/apps/api-core/src/lib/contentHash.ts`	Order-independent SHA-256 question fingerprint generator.
`/apps/api-ai/app/main.py`	Python FastAPI microservice for 3-Tier Cached RAG Study Notes, Part-II Descriptive Evaluation, and Daily Current Affairs.
`/packages/shared-types/src/index.ts`	Shared TypeScript interfaces, WBPSC exam presets, CSV parsers, and Zod validation schemas.